

Introduction to Python/SageMath

Minoru Hirose

いくつかの選択肢がある.

- ▶ オンライン上で実行する

- ▶ SageMathCell (<https://sagecell.sagemath.org/>)

- ▶ Cocalc (<https://cocalc.com/>)

- ▶ インストールして実行する

高性能のコンピュータを所有している場合は、インストールして実行を推奨する.
インストールはトラブルに見舞われやすく、情報もすぐに古くなりがち.

困ったら詳しくそうな人に相談するのがおすすめ.

見よう見まねでやってみる.

Python の公式ドキュメントを読む

en: <https://docs.python.org/ja/3/>

jp: <https://docs.python.org/3/>

公式のチュートリアルがおすすめ. 大事なことは大抵書いてある.

Sage の公式リファレンスを読む

<https://doc.sagemath.org/html/en/reference/index.html>

今日の話の構成

- ▶ インストールおよび基本的な使い方について
- ▶ ミニマリストのための Python/SageMath 入門
- ▶ Python/SageMath 入門

SageMath については殆ど解説しない.

ミニマリストのための Python/SageMath 入門

以下が分かれば（原理的には）何でも計算できる．

- ▶ print 文
- ▶ 代入文
- ▶ if 文
- ▶ while 文
- ▶ list

`print(...)` で式を出力できる.

code:

```
print( 7 )  
print( 2+10 )
```

output:

```
7  
12
```

ミニマリストのための Python/SageMath - print 文

式には四則演算 (`+`, `-`, `*`, `/`) に加え `**` (冪乗), `//` (割り算の整数部分), `%` (余り) が使用可能.

```
print( 100 ** 7 )  
print( 100 // 7 )  
print( 100 % 7 )
```

```
10000000000000000  
14  
2
```

必要があれば式を括弧 `( )` で囲む.

```
print( (1+2)*5 )
```

```
15
```


変数に計算結果を入れることが出来る.

```
a = 3 + 4  
print( a*a )
```

```
49
```

変数の値は途中で変更できる.

```
a = 3  
print( a )  
a = a + 5  
print( a )
```

```
3
```

```
8
```

if 文では，条件分岐できる

```
if 10<5:  
    print(1)  
    print(2)  
else:  
    print(3)
```

3

if 文の次の行の字下げは必須．普通は空白 4 文字．

条件式には `<`, `>`, `<=`, `>=`, `==`, `!=`, 等が使用できる。また `and`, `or`, `not` を用いて条件を組み合わせることも可能。

```
if 1<2 and 3!=4:  
    print(5)
```

```
5
```

while 文は条件が真の間、実行を繰り返す.

```
i = 0
while i < 5:
    print(i)
    i = i + 1
```

```
0
1
2
3
4
```

以下のプログラムは 100 以下の素数を全て出力する.

```
n = 1
while n<=100:
    num_of_divisors = 0
    i = 1
    while i<=n:
        if n%i==0:
            num_of_divisors = num_of_divisors + 1
        i = i + 1
    if num_of_divisors==2:
        print(n)
    n = n + 1
```

リストは並びを扱うためのデータ構造である。

リスト `x` に対し, `x[i]` で i 番目の要素を表す. (最初の要素が `x[0]`).

```
a = [1,10,100,1000]
print( a[2] )
a[1] = 7
print( a )
```

```
100
[1, 7, 100, 1000]
```

`+` でリストの連結が, `*` でリストの繰り返しができる

```
print( [1,2]+[3,4] )
print( [1,5]*3 )
```

```
[1, 2, 3, 4]
[1, 5, 1, 5, 1, 5]
```

ミニマリストのための Python/SageMath - List の使用例

$\Delta = q \prod_{n=1}^{\infty} (1 - q^n)^{24} = q - 24q^2 + 252q^3 - 1472q^4 + \dots$ の係数を求める.

```
N = 10
a = [1] + [0] * N
n = 1
while n<=N:
    s = 0
    while s<24:
        s = s+1
        i = N
        while i>=n:
            a[i] = a[i] - a[i-n]
            i = i - 1
        n = n+1
print(a)
```

[1, -24, 252, -1472, 4830, -6048, -16744, 84480, -113643, -115920, 534612]

ここまで紹介した機能を使うだけで原理的にはあらゆる計算が可能である（チューリング完全）。しかし、より多くの機能を知ることにより少ない労力で間違いのないプログラムを書くことが可能になる。

100 以下の素数を全て出力するプログラム。

```
for n in range(1,101):
    divisors = [i for i in range(1,n+1) if n%i==0]
    if len(divisors)==2:
        print(n)
```

Δ の係数を求めるプログラム。

```
a = [1] + [0]*10
for n in range(1,len(a)):
    for _ in range(24):
        a = [a[i]-a[i-n] if i>=n else a[i] for i,_ in enumerate(a)]
print(a)
```


Python/SageMath 入門

以下の項目について解説する.

- ▶ 行構造について
- ▶ 代入文
- ▶ 基本データ型 (int, str, bool, list, tuple, set, dict)
- ▶ フロー制御 (if, for)
- ▶ 関数定義
- ▶ リスト内包表記
- ▶ クラス
- ▶ デコレータ
- ▶ assert 文
- ▶ 標準モジュール

Python プログラムは、基本的に一番上から一行ずつ実行される。
1 行が長すぎる場合は `\` を使う。

```
a = 1000000 + \  
    1000000
```

```
print(a)
```

```
2000000
```

\ が不要なケースもある.

```
a = [1,2,3,4,  
      5,6,7,8]  
print(a)
```

```
[1,2,3,4,5,6,7,8]
```

en:

https://docs.python.org/3/reference/lexical_analysis.html#line-structure

jp:

https://docs.python.org/ja/3/reference/lexical_analysis.html#line-structure

各行の `#` 以降はコメントとして扱われる.

```
n = 37  # irregular prime
```

`a += b` は, `a=a+b` と同じ意味.

```
a = 3  
a += 5  
print(a)
```

8

```
a = 2  
b = 3
```

は

```
a, b = 2, 3
```

ともかける.

Python で扱われるオブジェクトには全て型がある.
型は `type` 関数で確認できる.

```
print( type([1,2,3]) )
```

```
<class 'list'>
```

基本データ型（一覧）

python($\neq$ SageMath) における基本的な型は以下の通り

- ▶ int
- ▶ float
- ▶ str
- ▶ bool
- ▶ list
- ▶ tuple
- ▶ set
- ▶ dict

int 型と float 型は SageMath ではあまり使われない.

基本データ型 (int)

int 型は整数を扱うための型.

```
type(123)
```

output by Python:

```
<class 'int'>
```

2 項演算

```
print( 100 + 3 )    # 103
print( 100 - 3 )    # 97
print( 100 * 3 )     # 300
print( 100 / 3 )     # 33.333333333333336 (Python case)
print( 100 // 3 )    # 33
print( 100 % 3 )     # 1 ( % means modulo operation)
print( 100 ** 3 )    # 1000000
```

数学研究にはあまり使わない2項演算

```
print( 12 ^ 6 )      # 10 (bitwise XOR, Python case)
print( 12 & 6 )      # 4  (bitwise AND)
print( 12 | 6 )      # 14 (bitwise OR)
print( 100 << 3 )    # 800 (bitshift)
print( 100 >> 3 )    # 12 (bitshift)
```

基本データ型 (int)

python の int には，以下の不満点がある．

- ▶ 割り算の結果が（分数ではなく）小数になる．
- ▶ `^` が冪乗ではない．

SageMath では，この不満点を解消する次のような方法が導入されている．

- ▶ 整数を扱うための新たな型である `Integer` を導入．
 - ▶ `Integer` 同士の割り算は分数として計算される．
 - ▶ その他にも大幅に機能が追加．
- ▶ 整数リテラルが含まれるコードを実行前に変換．(preparse)
 - ▶ `123+456` $\Rightarrow$ `Integer(123)+Integer(456)`
 - ▶ `123R+456R` $\Rightarrow$ `123+456`
- ▶ `~` が含まれるコードも実行前に変換．
 - ▶ `a^b` $\Rightarrow$ `a**b`
 - ▶ `a^^b` $\Rightarrow$ `a^b`

- ▶ `preparse` されたコードを確認する手っ取り早い方法は、エラーメッセージを見ること.
- ▶ コードの変換は、`import` されたコードには適応されない.
- ▶ `len` 等の python の標準関数は `int` を返す.
- ▶ `int` と `Integer` の演算結果は `Integer` になる.

基本データ型 (str)

`str` は文字列を扱うための型.

文字列を作るには `"` または `'` で囲む.

```
print( "abc" )  
print( 'Hello' )  
print( "Let's start" )
```

```
abc  
Hello  
Let's start
```

`"""` または `'''` で囲む場合もある.

基本データ型 (str) - フォーマット文字列

`f"..."` でフォーマット文字列が使える.

```
for i in (1..5):  
    print(f"2^2^{i} is equal to {2^2^i}")
```

```
2^2^1 is equal to 4  
2^2^2 is equal to 16  
2^2^3 is equal to 256  
2^2^4 is equal to 65536  
2^2^5 is equal to 4294967296
```

基本データ型 (str) - フォーマット文字列

その他の例.

```
for i in (1..5):  
    print(f"2^2^{i} is equal to {2^2^i:15}")
```

```
2^2^1 is equal to          4  
2^2^2 is equal to         16  
2^2^3 is equal to        256  
2^2^4 is equal to       65536  
2^2^5 is equal to     4294967296
```

```
a = 3  
print(f"{a=}")      # convenient for debug  
print(f"{1.0/a:.4}")
```

```
a=3  
0.3333
```

文字列型については他に多くのトピックがあるが省略する。
完全な情報は以下から見れる。

Python ドキュメント - ライブラリーリファレンス - 組み込み型 - テキストシーケンス型
en: <https://docs.python.org/3/library/stdtypes.html#textseq>
jp: <https://docs.python.org/ja/3/library/stdtypes.html#textseq>

基本データ型 (bool)

bool 型は True もしくは False を表す

```
print( 2 == 3 )  
print( type(2==3) )
```

```
False  
<class 'bool'>
```

基本データ型 (bool) - いくつかの注意点

Python では `a<b<c` のような書き方ができる。これは `a<b and b<c` と同値である。

en: <https://docs.python.org/3/reference/expressions.html#comparisons>

jp: <https://docs.python.org/ja/3/reference/expressions.html#comparisons>

`x and y` は `x` が True のとき `y` は評価されない。`x or y` も同様に `x` が False のとき `y` は評価されない。

```
0>1 and print("A")
```

```
0>1 or print("B")
```

```
B
```

`x if b else y` は `b` が真の時 `x`, `b` が偽のとき `y` となる。(3 項演算子)

```
3 if True else 5
```

```
3
```

```
3 if False else 5
```

```
5
```

基本データ型 (list)

list は列を扱うための型。要素の型は何でもいい。list を作るには `[ ]` で囲む。

```
[1, 2, "abc", [3, 4], [], True]
```

基本データ型 (list) - 成分へのアクセス

リスト `x` に対し, `x[i]` で i 番目の要素を表す. 負の数も指定可能である.

```
print(["a","b","c","d","e"][1])  
print(["a","b","c","d","e"][-1])
```

```
b  
e
```

基本データ型 (list) - スライス

リスト `x` に対し, `x[i:j]` で `x` の `i` から `j` までのスライスを表す.

```
[10, 20, 30, 40, 50][1:3]
```

```
[20, 30]
```

基本データ型 (list) - スライス

添字とスライスを理解するには次のチートシートが分かりやすい。

<https://wiki.python.org/moin/MovingToPythonFromOtherLanguages>

Python indexes and slices for a six-element list.

Indexes enumerate the elements, slices enumerate the spaces between the elements.

Index from rear:	-6	-5	-4	-3	-2	-1		a=[0,1,2,3,4,5]	a[1:]==[1,2,3,4,5]
Index from front:	0	1	2	3	4	5		len(a)==6	a[:5]==[0,1,2,3,4]
	+---+---+---+---+---+---+							a[0]==0	a[:-2]==[0,1,2,3]
	a b c d e f						a[5]==5	a[1:2]==[1]	
	+---+---+---+---+---+---+							a[-1]==5	a[1:-1]==[1,2,3,4]
Slice from front:	:	1	2	3	4	5	:	a[-2]==4	
Slice from rear:	:	-5	-4	-3	-2	-1	:		
									b=a[:]
									b==[0,1,2,3,4,5] (shallow copy of a)

str 型や tuple 型についても同様である。

基本データ型 (list) - step を指定するスライス

リスト s に対し, `s[i:j:k]` で s の i から j までの k 毎のスライスを表す.

```
[10, 20, 30, 40, 50][::-1]
```

```
[50, 40, 30, 20, 10]
```

基本データ型 (list) - len 関数

len(x) は x の長さを返す.

```
len([10,20,30])
```

```
3
```

len の返り値は int 型であることに注意.

```
print(type( len([10,20,30])) )  
print( len([10])/len([10,20,30]) )  
print( 1*len([10])/len([10,20,30]) )
```

```
<class 'int'>  
0.3333333333333333  
1/3
```


基本データ型 (list) - その他の関数, 演算, メソッド

コード	意味
<code>x in s</code>	<code>s[i]==x</code> となる i が存在するかどうか
<code>x not in s</code>	<code>x in s</code> の否定
<code>min(s)</code>	s の最小の要素
<code>max(s)</code>	s の最大の要素
<code>sorted(s)</code>	s の並び替え
<code>s.append(x)</code>	s の末尾に x を追加
<code>s.pop()</code>	s の末尾の要素を取り出し, また取り除く.
<code>s.sort()</code>	s を並び替える.

基本データ型 (tuple)

tuple 型は list 型と同様に列を扱うための型である.

tuple を作るには () で囲む.

```
t = (2,3,4)
print(type(t))
```

```
<class 'tuple'>
```

tuple 型は要素の追加や変更ができない.

```
t = (2,3,4)
t[0] = 5
```

```
-----
TypeError                                Traceback (most recent call last)
Input In [1], in <cell line: 2>()
      1 t = (Integer(2),Integer(3),Integer(4))
----> 2 t[Integer(0)] = Integer(5)

TypeError: 'tuple' object does not support item assignment
```

基本データ型 (set)

set は有限集合を扱うための型である.

```
s = set([2,3,5])  
s.add(7)  
print(s)  
print(2 in s)  
print(4 in s)
```

```
{2, 3, 5, 7}  
True  
False
```

基本データ型 (dict)

dict は連想配列を扱うための型である。連想配列とは、数学でいう定義域が有限集合となる関数のことである。

```
m = {}  
m[0] = 3  
m["a"] = 7  
print(m)  
print(m["a"])
```

```
{0: 3, 'a': 7}  
7
```

基本データ型 (dict)

dict 型は簡単な計算には不要かもしれないが、込み入ったアルゴリズムを実装する際には頻繁に使う。

以下は、有理数の乗法的独立性を判定する関数の実装例である。

```
def is_multiplicatively_independent(values):
    primes = set(prime for value in values for prime, _ in factor(value))
    ind_from_prime = {}
    for ind, prime in enumerate(primes):
        ind_from_prime[prime] = ind
    def get_vector(value):
        a = [0] * len(primes)
        for prime, n in factor(value):
            a[ind_from_prime[prime]] += n
        return vector(a)
    mat = matrix(get_vector(value) for value in values)
    return mat.rank() == len(values)

print( is_multiplicatively_independent([4,6,9]) )
```

False

プログラムの実行の流れをフローという。フローを制御するための構文として以下がある。

- ▶ if 構文
- ▶ for 構文
- ▶ while 構文
- ▶ try-except 構文（例外処理に使う）
- ▶ match 構文（新しい機能）

ここでは上2つについて解説する。

フロー制御 - if 文

if 文には 0 個以上の elif 部および、最後に高々 1 個の else 部をつけることができる。

```
for n in (1..6):  
    if n==1:  
        print(f"{n} is unit")  
    elif n.is_prime():  
        print(f"{n} is prime")  
    else:  
        print(f"{n} is composite")
```

```
1 is unit  
2 is prime  
3 is prime  
4 is composite  
5 is prime  
6 is composite
```

for はシーケンス型 (list, tuple, str 等) にわたって反復を行う。

```
for n in [2,3,7]:  
    print(n*n)
```

```
4  
9  
49
```


フロー制御 - for 構文 - range 関数

for 文と組み合わせると便利な関数として `range` がある.

```
for n in range(3):  
    print(n)
```

```
0  
1  
2
```

```
for n in range(2,5):  
    print(n)
```

```
2  
3  
4
```

```
for n in range(10,8,-1):  
    print(n)
```

```
10  
9  
8
```

`range` は `int` の列を返すことに注意する。 `srange` や `IntegerRange` は `Integer` の列を返す。

```
for n in range(3):  
    print(n, type(n))
```

```
0 <class 'int'>  
1 <class 'int'>  
2 <class 'int'>
```

```
for n in srange(3):  
    print(n, type(n))
```

```
0 <class 'sage.rings.integer.Integer'>  
1 <class 'sage.rings.integer.Integer'>  
2 <class 'sage.rings.integer.Integer'>
```

SageMath では次のような書き方も可能である

```
for n in (1..3):  
    print(n, type(n))
```

```
1 <class 'sage.rings.integer.Integer'>  
2 <class 'sage.rings.integer.Integer'>  
3 <class 'sage.rings.integer.Integer'>
```

```
for n in (1,3..9):  
    print(n)
```

```
1  
3  
5  
7  
9
```

`(a..b)` といった書き方は Pure Python では文法エラーになる。これらの機能は `preparse` によって実現されている。

```
(2..7)
```

⇒

```
(ellipsis_iter(Integer(2), Ellipsis, Integer(7)))
```

フロー制御 - for 構文 - break, continue

break 文は for ループ (または while ループ) を抜け出す。

```
for n in (1000..2000):  
    if n.is_prime():  
        print(n)  
        break
```

1009

continue 文はループの先頭に戻ってやり直す。

```
for n in (1..6):  
    if n.is_prime():  
        continue  
    print(n)
```

1

4

6

for 文の else 節は break されずにループを抜けた場合に実行される.

```
a,b = 14,16
for n in (a..b):
    if n.is_prime():
        print(f"{n} is the smallest prime between {a} and {b}")
        break
else:
    print(f"There are no primes between {a} and {b}")
```

There are no primes between 14 and 16

for-else の使用を非推奨としている文献も多い,

フロー制御 - for 構文 - enumerate と zip

for 文と組み合わせると便利な関数として `enumerate` と `zip` がある.

```
seq = [5,7,9]
for i,e in enumerate(seq):
    print(f"{i=}, {e=}")
```

```
i=0, e=5
i=1, e=7
i=2, e=9
```

```
seq1 = [5, 7, 9]
seq2 = [10,20, 30]
for x,y in zip(seq1, seq2):
    print(f"{x=}, {y=}")
```

```
x=5, y=10
x=7, y=20
x=9, y=30
```

関数を新たに定義するには `def` 文を使う.

```
def f(n):  
    return n*n  
print(f(2) + f(3))
```

13

```
def g(a,b):  
    return a*b  
print(g(2,3))
```

6

```
def h():  
    pass  
print(h())
```

None

関数定義 - 変数のスコープ

関数内での変数への代入は、関数の外の変数に影響を与えない。

```
a = 5
def f():
    a = 3
    print(a, "Hello")
f()
print(a)
```

```
3 Hello
5
```

関数定義 - 変数のスコープ

関数内で、外の変数のメソッドを呼び出したりは可能.

```
x = []  
def f(n):  
    x.append(n)  
f(3)  
x
```

```
f()
```

上で定義された `f` のように、状態の変化を引き起こす関数を“副作用”をもつ関数という.
副作用は最小限に抑えるのがバグを減らす原則.

関数定義 - キーワード引数

```
def f(a,b,c):  
    print(f"{a=}, {b=}, {c=}")
```

```
f(2, c = 5, b = 4)
```

```
a=2, b=4, c=5
```

- ▶ プログラムの可読性が向上する.
- ▶ 引数が順不同になる.

関数定義 - デフォルト引数

```
def f(a,b=7):  
    return a+b  
print( f(10) )  
print( f(10, 10) )
```

17

20

関数定義 - 可変長引数

```
def f(v,*args, **keywords):  
    print(f"{args=}")  
    print(f"{keywords=}")  
f(1,2,3,a=4,b=5)
```

```
args=(2, 3)  
keywords={'a': 4, 'b': 5}
```

`help(func)` で `func` の説明を見ることができる.

```
help(max)
```

```
Help on built-in function max in module builtins:
```

```
max(...)
```

```
max(iterable, *[, default=obj, key=func]) -> value
```

```
max(arg1, arg2, *args, *[, key=func]) -> value
```

With a single iterable argument, return its biggest item. The default keyword-only argument specifies an object to return if the provided iterable is empty.

With two or more arguments, return the largest argument.

関数定義 - 再帰とメモ化再帰

```
def fib(n):  
    return n if n <= 1 else fib(n-1) + fib(n-2)  
fib(10)
```

55

```
from functools import cache  
@cache  
def fib(n):  
    return n if n <= 1 else fib(n-1) + fib(n-2)  
fib(100)
```

354224848179261915075

関数定義 - 関数内関数

関数定義の内部で別の関数を定義することもできる.

```
def f(a,b,c,d):  
    p = 3  
    def s(x):  
        return x**p  
    return s(a)+s(b)+s(c)+s(d)  
f(1,2,3,4)
```

100

関数の合成を返す関数

```
def compose(f,g):  
    def h(x):  
        return f(g(x))  
    return h  
compose(factorial, factorial)(3)
```

720

以下については公式チュートリアルで詳しく説明されている。

- ▶ 位置専用引数, キーワード専用引数. `def f(a,b,/,c,d,*,e,f):`
- ▶ ラムダ式. `lambda x: x+3`
- ▶ ドキュメンテーション文字列. (関数の詳細をコメントする. `help` 関数等で活用される.)
- ▶ 関数のアノテーション (型ヒント)

リスト内包表記

リストを作る便利な方法としてリスト内包表記がある.

```
print( [x*x for x in [1,2,3]] )  
[1, 4, 9]
```

if も使用可能である.

```
print( [x*x for x in [1,2,3] if x.is_prime()] )  
[4, 9]
```

for をネストすることもできる.

```
print( [y for x in [1,2,3,4] for y in x.divisors() ] )  
[1, 1, 2, 1, 3, 1, 2, 4]
```

クラス - インデックスのクラス

`class` 構文で新たな型を作ることができる.

```
class Index:
    def __init__(self, seq):
        self.seq = seq
    def weight(self):
        return sum(self.seq)
    def depth(self):
        return len(self.seq)

a = Index([2,3,4])
print( a.weight(), a.depth() )
```

9 3

上の例では `Index` が“クラス”, `a` が“インスタンス”, `a.weight` が“メソッド”と呼ばれる.

デフォルトではインスタントを表示してもあまりいい感じではない。

```
class Index:
    def __init__(self, seq):
        self.seq = seq
print( Index([2,3,4]) )
```

```
<__main__.Index object at 0x136ee0880>
```

`__repr__` メソッドを定義することで表示をコントロールできる。

```
class Index:
    def __init__(self, seq):
        self.seq = seq
    def __repr__(self):
        return f"Index{tuple(self.seq)}"
print( Index([2,3,4]) )
```

```
Index(2, 3, 4)
```

クラス - 特殊メソッド

`__init__` や `__repr__` のように特別な意味を持ったメソッドを特殊メソッドという。Python には 100 近くの特特殊メソッドがある。特殊メソッドは全て `--` で始まり `--` で終わる。以下は、特殊メソッドの例である。

メソッド	意味
<code>__init__(self)</code>	インスタンスの生成時に呼ばれる
<code>__repr__(self)</code>	表示に使われる文字列を返す
<code>__eq__(self)</code>	<code>self == other</code>
<code>__hash__(self)</code>	ハッシュ関数（辞書のキーにする際に使う）
<code>__bool__(self)</code>	真偽値
<code>__getitem__(self, key)</code>	<code>self[key]</code>
<code>__iter__(self)</code>	イテレータ。（ <code>for _ in self</code> のような書き方が可能になる）
<code>__add__(self, other)</code>	<code>self + other</code>

詳しくは言語リファレンス - 3. データモデル - 3.3. 特殊メソッド名 を参照。
<https://docs.python.org/3/reference/datamodel.html#specialnames>

dataclass デコレータを使うと楽にクラスが作れる.

(`__init__`, `__repr__`, `__eq__` 等を自動で生成)

```
from dataclasses import dataclass
@dataclass
class Pair:
    a: int
    b: int
print( Pair(2,3) )
print( Pair(2,3)==Pair(2,3) )
print( Pair(2,3)==Pair(2,4) )
```

```
Pair(a=2, b=3)
True
False
```

クラスの機能を引き継いで新たなクラスを作れる.

```
class Index:
    def __init__(self, seq):
        self.seq = seq
    def weight(self):
        return sum(self.seq)
    def depth(self):
        return len(self.seq)

class IndexForEulerSum(Index):
    def weight(self):
        return sum(abs(u) for u in self.seq)

a = IndexForEulerSum([-3,3])
print(a.depth(), a.weight())
```

2 6

インスタンスのメソッドを調べるには `dir` 関数が便利.

```
dir(3)
```

これを実行すると 3 のメソッド全て (300 近く) 表示される.

```
['__abs__',  
 '__add__',  
 ...  
 'is_prime_power',  
 ...  
 'xgcd']
```

`_` で始まるメソッドは外部から使われることを想定していない. (プライベートメソッド)
気になるメソッドがあれば, `help(3.is_prime_power)` などして使用法を調べる.


```
@deco  
def func():  
    (...)
```

は、次と同じ意味.

```
def func():  
    (...)  
func = deco(func)
```

主に関数の機能強化のために使われる.
クラスにも使用可能.

`assert condition` で `condition` が偽のときエラーが発生する。デバッグに便利。プログラムのあちこちに書くことを推奨する。

```
assert "a"=="b"
```

```
-----  
AssertionError                                Traceback (most recent call last)  
Input In [1], in <cell line: 1>()  
----> 1 assert "a"=="b"
```

```
AssertionError:
```

標準モジュール

モジュールとは機能を集めたもの。Python に初めから搭載されているモジュールを標準モジュールという。モジュールの詳細は、公式チュートリアルで解説されている。自作モジュールを単に import した場合、preparse されないことに注意。ここではよく使う標準モジュールの機能を紹介する。

再帰回数の上限を変更。

```
import sys
sys.setrecursionlimit(1000000)
```

時間の計測。

```
from time import time
start = time()
sum([k**4 for k in (1..10**6)])
stop = time()
print(f"execution_time = {stop - start}")

execution_time = 0.8384356498718262
```

乱数の生成

```
from random import random, randint
print([random() for _ in range(3)]) # real numbers between 0 and 1
print([randint(1,3) for _ in range(10)]) # integers between 1 and 3
```

```
[0.5289727734272919, 0.4469770781937602, 0.429926328178893]
[2, 3, 2, 3, 1, 3, 3, 2, 2, 1]
```

よく使う標準モジュール

メモ化

```
from functools import cache
@cache
def fib(n):
    return n if n <= 1 else fib(n-1) + fib(n-2)
fib(100)
```

```
354224848179261915075
```

データクラス

```
from dataclasses import dataclass
@dataclass
class Pair:
    a: int
    b: int
Pair(2,3)
```

```
Pair(a=2, b=3)
```

End